

Software Design Description

SQL Code Generator

Afrem Jasmine-Emilia si Bimbirica Andra
IR3 2026, grupa 2

Contents

1	Arhitectura Generala	2
2	Module	2
3	Stare Partajata	2
4	Protocol	3
5	Fluxuri Interne	4
5.1	Upload ER	4
5.2	Generare SQL	4
5.3	Validare Insert	4
6	Validare Inserturi	5
7	Configurare	5
8	Administrare	6
9	Specificatie Web/API	6
10	Calitate Si Verificare	6

1 Arhitectura Generala

Sistemul are trei executabile:

- **server**: proces UNIX C care mentine starea, accepta conexiuni si proceseaza cereri.
- **client**: CLI pentru upload schema ER, generare SQL si validare inserturi.
- **admin**: client ncurses pentru rapoarte administrative.

Serverul expune doua socket-uri TCP:

- portul ordinar pentru clienti care trimit scheme si inserturi;
- portul admin pentru rapoarte.

Ambele socket-uri si conexiunile active sunt multiplexate cu `poll`. Cererile clientilor ordinari sunt puse intr-o coada FIFO comuna. Un thread worker scoate cererile din coada si trimite raspunsul sincron catre clientul care a initiat cererea.

2 Module

- `src/protocol.c`: conectare TCP, listen TCP, citire/scriere completa si serializare mesaje.
- `src/model.c`: model ER, parser JSON minimal, generator SQL, parser/validator INSERT, rapoarte admin.
- `src/server.c`: socket-uri, `poll`, coada FIFO, thread worker, configurare libconfig, fork/waitpid.
- `src/client.c`: comenzi interactive si demo neinteractiv.
- `src/admin_client.c`: UI ncurses pentru rapoarte.

3 Stare Partajata

`SharedState` este alocata cu:

```
mmap(NULL, sizeof(SharedState), PROT_READ | PROT_WRITE,  
      MAP_SHARED | MAP_ANONYMOUS, -1, 0);
```

Structura contine:

- tabele, coloane si randuri acceptate;

- clienti conectati;
- contori de comenzi;
- istoric circular;
- adancimea cozii;
- starea conexiunii admin.

Procesele copil create pentru validarea inserturilor citesc aceeași stare partajată. Copilul nu modifică starea; doar întoarce rezultatul validării prin pipe. Parintele aplică batch-ul numai dacă validarea copilului reușește.

4 Protocol

Header-ul este fix:

```
typedef struct {
    uint32_t msg_size;
    uint32_t client_id;
    uint32_t op_id;
    uint32_t flags;
} MsgHeader;
```

Campurile sunt trimise în network byte order. Payload-ul are lungimea `msg_size` și nu folosește delimitatori suplimentari.

Operații:

- `OP_CONNECT = 0`
- `OP_BYE = 5`
- `OP_UPLOAD_BEGIN = 10`
- `OP_UPLOAD_CHUNK = 11`
- `OP_UPLOAD_END = 12`
- `OP_GENERATE_SQL = 20`
- `OP_VALIDATE_INSERT = 30`
- `OP_OK = 40`
- `OP_ERROR = 41`
- `OP_ADMIN_LOGIN = 100`
- `OP_ADMIN_REPORT = 101`
- `OP_ADMIN_BYE = 105`

5 Fluxuri Interne

5.1 Upload ER

1. Clientul cere `OP_UPLOAD_BEGIN`.
2. Serverul creeaza `data/upload_<client_id>.schema.json`.
3. Clientul trimite chunk-uri de maxim 64 KB.
4. Serverul append-uie fiecare chunk.
5. La `OP_UPLOAD_END`, serverul parseaza fisierul si reseteaza modelul curent.

5.2 Generare SQL

Generatorul parcurge tabelele din `SharedState` si construiesc instructiuni `CREATE TABLE`. Constraint-urile sunt scrise inline pe coloane:

- `PRIMARY KEY`;
- `NOT NULL`;
- `UNIQUE`;
- `REFERENCES table(column)`.

5.3 Validare Insert

1. Worker-ul primeste `OP_VALIDATE_INSERT`.
2. Incrementeaza contorul de inserturi.
3. Creeaza pipe si proces copil.
4. Copilul parseaza batch-ul si verifica toate constrangerile.
5. Copilul scrie statusul in pipe si iese cu `_exit`.
6. Parintele asteapta cu `waitpid`.
7. Parintele ruleaza din nou parsarea/aplicarea in procesul principal, pentru a modifica starea comuna.

Acest design separa validarea demonstrativa in proces copil, dar pastreaza mutatiile in parinte, evitand modificari accidentale facute de copil.

6 Validare Inserturi

Parserul accepta:

```
INSERT INTO table (col1, col2) VALUES (v1, v2), (v3, v4);
```

Se verifica:

- tabela exista;
- coloanele exista;
- numarul de coloane corespunde cu numarul de valori;
- NOT NULL si PRIMARY KEY nu primesc NULL;
- PRIMARY KEY si UNIQUE nu se repeta in starea existenta sau in acelasi batch;
- foreign key-ul indica o valoare existenta in tabela referita sau intr-un rand validat anterior din acelasi batch.

7 Configurare

Serverul integreaza libconfig. Valorile sunt incarcate in `RuntimeConfig`.

Ordinea de precedenta:

1. valori implicite: `SQLCG_PORT`, `SQLCG_ADMIN_PORT`, 60;
2. fisier libconfig, implicit `config.cfg`;
3. variabile de mediu: `CONFIG_PATH`, `SERVER_PORT`, `ADMIN_PORT`, `ADMIN_TIMEOUT`;
4. argumente CLI: `-config`, `-port`, `-admin-port`, `-admin-timeout`.

Daca fisierul de configurare lipseste sau nu poate fi citit, serverul afiseaza un mesaj de fallback si continua cu valorile disponibile.

Makefile-ul activeaza calea libconfig cand `pkg-config` gaseste biblioteca si defineste `HAVE_LIBCONFIG`. Pentru sisteme de dezvoltare fara headerul `libconfig.h`, exista un parser de fallback limitat la cheile folosite in `config.cfg`, astfel incat restul demo-ului poate fi compilat si testat.

8 Administrare

Clientul admin foloseste ncurses si cere rapoarte prin `OP_ADMIN_REPORT`.

Categorii implementate:

- `clients`: clienti ordinari conectati si idle time;
- `commands`: comenzi totale, inserturi si esecuri;
- `avg`: durata medie de executie;
- `history`: istoric recent;
- `tables`: tabele incarcate si numar de randuri;
- `queue`: adancimea cozii FIFO.

Serverul permite un singur admin conectat. Un al doilea admin primeste eroare pana cand primul se deconecteaza sau expira prin timeout.

9 Specificatie Web/API

Executabilul curent foloseste protocol TCP binar. `docs/openapi.yaml` documenteaza o interfata HTTP/WS echivalenta pentru etapa Web Service:

- `POST /schemas` pentru incarcare schema;
- `POST /sql/generate` pentru generare SQL;
- `POST /inserts/validate` pentru validare insert;
- `GET /admin/reports/{category}` pentru rapoarte;
- `/ws/protocol` pentru mesaje WebSocket bazate pe aceleasi operatii.

Aceasta separare pastreaza protocolul cerut pentru Milestone 1 si lasa clar contractul pentru extensia de nivel C.

10 Calitate Si Verificare

Build-ul foloseste:

```
CFLAGS=-g -Wall -Wextra -std=c11 -D_DEFAULT_SOURCE -Iinclude
LDFLAGS=-pthread $(LIBCONFIG_LIBS)
NCURSES_LIBS=-lncurses -ltinfo
```

Makefile-ul detecteaza libconfig prin pkg-config. Pentru prezentarea Milestone 1, biblioteca trebuie sa fie instalata, astfel incat serverul sa fie compilat cu `-DHAVE_LIBCONFIG` si sa foloseasca API-ul `config_t`, `config_read_file`, `config_lookup_int`.

Demo recomandat:

```
make
./server
./client --input examples/er_schema.json --generate \
  --insert-file examples/inserts_ok.sql --no-repl
./client --input examples/er_schema.json \
  --insert-file examples/inserts_fail_fk.sql --no-repl
./admin
```

Verificari recomandate inainte de predare:

```
make -B all
clang-tidy src/*.c -- -std=c11 -D_DEFAULT_SOURCE -Iinclude
```

Codul este documentat cu antete de fisier si comentarii explicative in modulele principale. Build-ul trebuie sa ramana fara warning-uri pe optiunile din Makefile.